CT087-3-3 Real-Time Systems

Real-Time Communication Patterns (UDP & MQTT)

A·P·U
ASIA PACIFIC UNIVERSITY
OF TECHNOLOGY & INNOVATION

TOPIC LEARNING OUTCOMES

- By the end of this session you will be able to:
  - Discuss the difference types of RT communication patterns
  - Explain the benefits and limitations of UDP and MQTT in the context of hard and soft RT systems

Contents & Structure

- Theory of Real-Time Networking
- Latency, Jitter, and Determinism.
- The "Hard" vs. "Soft" Real-Time Network boundary.
- Hard Real-Time (UDP)
- Why TCP is the enemy of determinism.
- Implementing raw UDP in tokio.
- Soft Real-Time (MQTT)
- The Publish-Subscribe Model.
- QoS Levels (0 vs 1 vs 2) and their impact on latency.

Recap from the last lecture

- A Real-Time System (RTS) is defined by temporal correctness (meeting deadlines).
- "Real-Time" ≠ "Fast." It means "Predictable."\
- Categories: Hard (catastrophic failure), Firm (worthless result), Soft (degraded quality).
- Rust's Potential: No GC (predictable cleanup), memory safety, and fearless concurrency.

Defining "Real-Time" in Networking

- Standard Networking: "Correctness is more important than timeliness" (e.g., loading a webpage).
- Real-Time Networking: "Timeliness is part of correctness."
- Key Metrics:Latency:
  - Time from Source → Destination.
  - Jitter: Variance in latency. (High jitter = unpredictable control loops).
  - Deadline Miss: If a packet arrives after t, is it still useful?
    - Hard RT: No (System failure).
    - Soft RT: Yes (Degraded quality).

Transport Layer: UDP vs. TCP

- *How should data be delivered from one program to another across a network?*
- TCP and UDP are the two main transport-layer protocols used for sending data over a network (they sit on top of IP).
- TCP is a reliable, connection-oriented protocol. Before sending data, a connection is established (3-way handshake) and both sides agree on communication parameters.
- UDP is an unreliable, connectionless protocol. There is:
  - No handshake
  - No connection setup
  - No guarantee of delivery

Transport Layer: UDP vs. TCP

- TCP (Transmission Control Protocol):
  - Mechanism: Handshakes, retransmissions, ordered delivery.
  - The Problem: Head-of-Line Blocking. If Packet 1 is lost, Packet 2 waits. This causes unpredictable latency spikes (e.g., 200ms pause).
- UDP (User Datagram Protocol):
  - Mechanism: Fire and forget.
  - The Benefit: Lowest possible overhead. No connection state.
  - Verdict: The standard for Hard Real-Time (Voice, Video, Control Systems).

Transport Layer: UDP vs. TCP

- What TCP guarantees:
  - Reliable delivery (no data loss).
  - Ordered delivery (data arrives in sequence).
  - Error detection and retransmission.
  - Flow control (don't overwhelm receiver).
  - Congestion control (don't overwhelm network).
- How TCP behaves if a packet is lost:
  - TCP waits.
  - Retransmits the packet
  - All later packets are delayed until the missing one arrives (head-of-line blocking)
- Advantages:
  - Reliable
  - Easy to use
  - Ideal for data correctness.
- Disadvantages:
  - Unpredictable delays.
  - Latency spikes
  - High jitter
  - Not deadline-aware
- Typical TCP use examples: Email, HTTP/HTTPS, FTP, database connection

Transport Layer: UDP vs. TCP

- What UDP does not guarantee:
  - No reliability (possible data loss).
  - No ordering.
  - No retransmission.
  - No congestion control.
- How UDP behaves if a packet is lost:
  - It is simply lost.
  - The system continues with the next packet
- Advantages:
  - Very low latency
  - Minimal overhead
  - Predictable timing
  - No blocking
- Disadvantages:
  - Data may be lost
  - Data may arrive out of order
  - Application must handle reliability if needed
- Typical TCP use examples: Voice calls (VoIP), Video streaming, Online gaming, Sensor data, Robotics and control systems

Rust’s Async Ecosystem (tokio)

- Real-time networked systems in Rust rely on Asynchronous I/O.
- The Model: Rust uses a "Poll-based" model. Futures do nothing unless polled by a runtime.
- Why it matters for RTS:
  - Concurrency: One thread can handle thousands of sensor streams.
  - Low Overhead: No OS thread context switching for every packet.

Code: Async UDP in Rust

- Using tokio for a non-blocking UDP loop:

```rust
use tokio::net::UdpSocket;

#[tokio::main]
async fn main() -> std::io::Result<()> {
    let sock = UdpSocket::bind("0.0.0.0:8080").await?;
    let mut buf = [0; 1024];

    loop {
        // Await yields execution until a packet actually arrives
        let (len, addr) = sock.recv_from(&mut buf).await?;

        // Critical Section: Process immediately
        // In RTS, we avoid complex allocations here!
        println!("{} bytes from {:?}", len, addr);
    }
}
```

Serialization for Real-Time

- Sending JSON ({"temp": 20}) is easy but slow (parsing overhead).
- Zero-Copy Serialization:
- Libraries like `rkyv` or `bincode` allow us to cast raw bytes directly into Rust structs.
- Trade-off: Safety vs Speed. Rust ensures this is memory-safe, unlike C/C++.

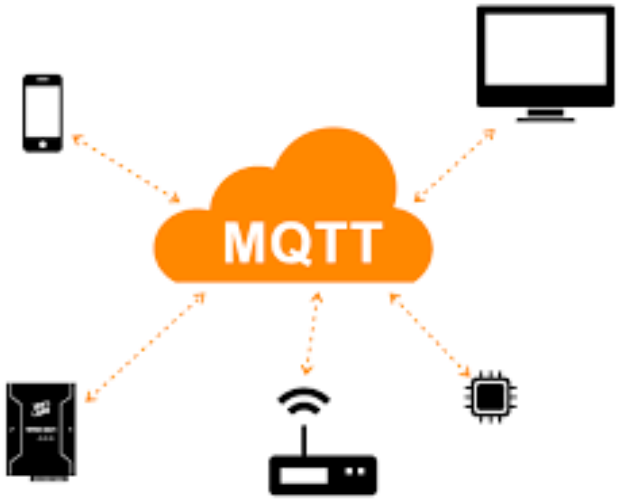
Format	Parse Time	Determinism
JSON	High (Variable)	Low
Protobuf	Medium	Medium
Bincode/Rkyv	Near Zero	High

Soft Real-Time: The Need for Middleware

- Direct UDP is great for a single wire, but hard for distributed systems.
- Eeal-time systems involved one sensor any many consumers.
- Scenario: A temperature sensor needs to send data to:
    1. The Database (Log).
    2. The Dashboard (UI).
    3. The Alarm System (Safety).
- Solution: A Message Broker (Publish/Subscribe Pattern).

Introduction to MQTT

- MQTT (Message Queuing Telemetry Transport):
  - The standard for IoT and Industrial Automation.
  - Runs over TCP (usually), but designed for unreliable networks.
- Architecture:
  - Publisher: The Sensor (sends data).
  - Broker: The Central Post Office (distributes messages, e.g., Mosquitto).
  - Subscriber: The Dashboard (received data).
- Why MQTT? Extremely lightweight headers (2 bytes minimum) compared to HTTP.



MQTT Quality of Service (QoS)

- The most critical setting for Real-Time performance.
- QoS 0 (At Most Once):
  - "Fire and Forget". No confirmation. Fastest.
  - Use Case: High-frequency sensor data (100Hz). If we miss one, the next one is coming in 10ms.
- QoS 1 (At Least Once):
  - Sender stores message until Broker replies "PUBACK".
  - Use Case: Alarms/Events.
- QoS 2 (Exactly Once):
  - 4-step handshake. Too slow for most RTS.

Rust & MQTT (rumqttc)

- We use `rumqttc`, a pure-Rust, high-performance MQTT client.
- Integrates natively with tokio.

```rust
use rumqttc::{MqttOptions, AsyncClient, QoS};

fn example(){

    let mut options = MqttOptions::new("sensor_1", "localhost", 1883);
    // Keep alive is vital for detecting broken lines
    options.set_keep_alive(Duration::from_secs(5));

    let (client, mut eventloop) = AsyncClient::new(options, 10);
}
```

Code: The Event Loop

- In rumqttc, the event loop is explicit. This allows precise control over how we handle network events.

```
async fn example(){
    tokio::spawn(async move {
    loop {
        // Poll the event loop. This handles PINGs and RECONNECTs automatically.
        match eventloop.poll().await {
            Ok(notification) => {
                // Handle incoming messages or status updates
            },
            Err(e) => {
                eprintln!("Connection lost: {:?}", e);
                // In RTS, we might trigger a safety fallback here!
            }
        }
    }
});
}
```

Measurement & Metrics

- Requirement: Verify performance impact.
- How do we measure network latency in Rust?
- Tool: `std::time::Instant` (Monotonic Clock).
- Do NOT use: `SystemTime` (Wall clock). NTP updates can cause negative latency!

```rust
let start: Instant = Instant::now();
socket.send(data).await?;
socket.recv(ack).await?;
let rtt: Duration = start.elapsed(); // Precise duration
}
```

The "Dead Man's Switch" Pattern

- In safety-critical systems, Silence = Danger.
- We cannot assume the system is working just because there are no errors.
- Implementation:
  - Receiver expects a packet every 100ms.
  - If last_packet_time > 200ms -> Trigger Emergency Stop.
- We will implement this in the Lab using `tokio::time::timeout`.

Lab 8 Overview

- Goal: Build and Benchmark a Real-Time Pipeline.
- Tools: Rust, tokio, Mosquitto (Broker).
- Task 1: UDP Speed Test.
  - Build a "Ping Pong" server.
  - Use hdrhistogram to graph the latency distribution (Microseconds).
- Task 2: MQTT Telemetry.
  - Build a Sensor (Pub) and Dashboard (Sub).
  - Experiment with QoS 0 vs QoS 1.

Summary

- UDP: The "metal" layer. Use for tight control loops where every microsecond counts.
- MQTT: The "application" layer. Use for distributing data to multiple systems where small delays (<50ms) are acceptable.
- Rust: Provides the safety of memory management without the latency spikes of Garbage Collection.